Задача A. Много ракушек

Формальная **постановка задачи**: найти максимальную сумму подотрезка фиксированной длины (количества элементов).

Важно: обратите внимание, что в данной задаче сумма подотрезка может выходить за границы 32-битного целочисленного типа данных (**переполнение**). Используйте 64-битные целочисленные типы данных.

Сразу рассмотрим случай $K \geqslant N$: в таком случае ответом является **сумма всего массива**.

Далее будем считать, что $K < N$.

Общая идея решения: перебрать все отрезки длины K (их $O(N-K)$ штук) и **вычислить сумму** каждого.

1. **Суммирование циклом** (**неэффективно**)

   Если вычислять сумму каждого рассматриваемого отрезка за $O(K)$, то итоговая асимптотика получается $O((N-K) \cdot K)$.

   Данная функция представляет из себя направленную вниз параболу, поэтому её максимум достигается в вершине параболы $K = \frac{N}{2}$.

   Итоговая асимптотика в худшем случае $O((N - \frac{N}{2}) \cdot \frac{N}{2}) = O(N^2)$.

2. **Скользящее окно**.

   Вычислим сумму первых K элементов массива («окно») и инициализируем данным значением ответ.

   Пусть в данный момент окно задаёт полуинтервал $[i-K; i)$ и имеет сумму S.

   Следующее окно будет задаваться полуинтервалом $(i-K; i]$ — то есть будет иметь сумму $S - a_{i-K} + a_i$.

   Таким образом можно перемещать окно в порядке возрастания его границ, пересчитывая сумму в окне за $O(1)$.

   Итоговая асимптотика: $O(K + (N-K)) = O(N)$.

3. **Префиксные суммы**.

   Построим массив P_i — сумму первых i элементов массива:

   - $P_0 = 0$;
   - $P_i = P_{i-1} + a_i$.

   В таком случае сумма на полуинтервале $(i-K; i]$ будет равна $P_i - P_{i-K}$.

   Итоговая асимптотика: $O(N + (N-K)) = O(N)$

   Дополнительная информация:

   - Статья на Алгоритмике:
     https://ru.algorithmica.org/cs/range-queries/prefix-sum/
   - Курс на Codeforces EDU:
     https://codeforces.com/edu/course/3/lesson/10

Задача B. Выигрыш

Формальная **постановка задачи**: вычисление количества путей из вершины S в вершину E длиной ровно K ребер.

В данном конкретном случае $S = E = 1$.

Данная задача имеет **два решения** — фундаментально схожих, но отличающихся в деталях и асимптотике:

1. **Динамическое программирование**

   Пусть $dp_{q,v}$ — количество путей из вершины v в вершину E длиной ровно q рёбер:

   - $dp_{0,E} = 1$ — заканчиваем путь в вершине E единственным образом.
   - ответом является $dp_{K,S}$ — прошли K рёбер из вершины S.
   - вычисление будем производить в порядке возрастания величины q: $dp_{q,v} = \sum dp_{q-1,to}$ по всем рёбрам (v, to).

   Пояснение: все способы пройти из v в E через q рёбер **разделяются на способы**:

   (a) пройти через ребро (v, to);
   (b) пройти из to в E через $(q-1)$ ребро.

   Итоговая асимптотика: $O(K \cdot (N + M))$, где M — количество рёбер в графе.

   При $O(M) = O(N^2)$ получается $O(K \cdot N^2)$.

   Дополнительная информация:

   - Введение в динамическое программирование
     https://ru.algorithmica.org/cs/general-dynamic/

2. **Возведение матрицы в степень**

   Рассмотрим матрицу смежности A из N строк и N столбцов, где

   - $A_{i,j} = 1$, если в графе есть ребро (i, j).
   - $A_{i,j} = 0$ в ином случае.

   Пусть A^q — q-я степень матрицы A.

   В таком случае $A^q_{i,j}$ — количество путей из i в j длиной ровно q рёбер.

   Доказательство через ММИ:

   (a) A^1 содержит количество рёбер между вершинами — количество путей длины 1.
   (b) Пусть A^q содержит количество путей длиной ровно q рёбер.
   (c) Значение $A^{q+1}_{i,j}$ равно $\sum (A_{i,v} \cdot A^q_{v,j})$ по всем v.

   Пояснение: все способы пройти из i в j через $(q+1)$ ребро **разделяются на способы**:

   (a) пройти из i в v через одно ребро;
   (b) пройти из v в j через q рёбер.

   Ответом является значение $A^K_{S,E}$.

   Итоговая асимптотика: $O(N^3 \cdot log(K))$ при использовании **быстрого (бинарного)** возведения матрицы в степень.

   Дополнительная информация:

- Умножение матриц:
  https://ru.algorithmica.org/cs/algebra/matrix/
- Бинарное возведение на E-maxx:
  https://e-maxx.ru/algo/binary_pow
- Бинарное возведение на Алгоритмике:
  https://ru.algorithmica.org/cs/algebra/binpow/

Задача C. Дорога

Формальная **постановка задачи**: найти длину объединения отрезков на числовой прямой.

Данная задача решается с помощью способа **«сортировка событий»**.

Решение:

- Каждый отрезок $[L; R]$ на прямой задаётся двумя событиями:
  1. «начался» в позиции L;
  2. «закончился» в позиции R.
- Построим список всех событий и отсортируем его по возрастанию координаты события.
  - при равенстве координат события «начала» идут ранее событий «конца».
    (в общем случае правило сортировки событий с одинаковыми координатами может отличаться).
- Введём две величины:
  1. $last$ — координата последнего обработанного события.
  2. $started$ — количество начатых, но не закончившихся отрезков.
- Обработаем события в получившемся после сортировки порядке:
  - Пусть мы обрабатываем событие в координате X.
  - Если $started > 0$, то прибавляем к длине объединения величину $(X - last)$.
    (**хотя бы один** отрезок покрывает данный участок прямой).
  - Обновляем $last = X$.
  - Если событие «начала», то увеличиваем $started$ на 1, иначе – уменьшаем на 1.

Итоговая асимптотика: $O(N)$ на построение, $O(N \cdot log(N))$ на сортировку и $O(N)$ на обработку событий.

Суммарно $O(N) + O(N \cdot log(N)) + O(N) = O(N \cdot log(N))$.

Дополнительная информация:

- Дополнительные задачи на сканирующую прямую
  https://ru.algorithmica.org/cs/decomposition/scanline/

Задача D. Веселые фонари

Формальная **постановка задачи**: можно ли распределить вершины графа на два непересекающихся множества A и B таким образом, что все ребра соединяют вершины из разных множеств?

Определение: граф, удовлетворяющий вопросу задачи, называется **двудольным** (*bipartite*).

Соответственно, более **краткая постановка задачи**: проверить, является ли граф двудольным.

Одним из способов является **распределение через *обход***.

Определение: *обходом* графа называют процесс систематического просмотра всех вершин или рёбер графа, чтобы найти некоторые вершины, удовлетворяющие определённым условиям.

Решение:

- Каждая вершина графа может быть в одном из трёх состояний:
  1. не распределена (начальное состояние);
  2. распределена в множество A;
  3. распределена в множество B.
- Обработаем все вершины графа (в любом порядке):
  1. Пусть мы обрабатываем вершину s.
  2. Если вершина s уже распределена, то переходим к следующей вершине.
  3. Иначе распределяем вершину s в множество A, после чего запускаем из вершины s **модифицированный обход** графа.
- После обработки всех вершин **проверяем каждое ребро** графа на выполнение условия двудольности.

Рассмотрим принцип **модификации обхода** графа:

1. Пусть в процессе обхода мы рассматриваем ребро (v, u).
2. Если вершина u уже распределена, то переходим к следующему ребру.
3. Иначе **распределяем** вершину u в множество, **отличное** от множества вершины v, после чего обрабатываем ребро по стандартным правилам обхода.

Итоговая асимптотика: при использовании обхода в *глубину* или *ширину* $O(N + M)$, где N – количество вершин, M – количество рёбер.

Дополнительная информация:

- Введение в теорию графов на Codeforces EDU
  https://codeforces.com/edu/course/2/lesson/8
- Поиск в **глубину**
  https://ru.algorithmica.org/cs/graph-traversals/dfs/
- Поиск в **ширину**
  https://ru.algorithmica.org/cs/shortest-paths/bfs/

- Раскраски графов

  https://ru.algorithmica.org/cs/graph-traversals/bipartite/

Задача E. Старый шифр

Формальная **постановка задачи**: отсортируйте массив, используя описанное в условии правило сравнения (*компаратор*).

Современные языки, допустимые по правилам ICPC (C++, Java, Python и т.д.) содержат в своей библиотеке **встроенный алгоритм сортировки**, работающий за $O(N \cdot log(N))$.

Единственная сложная часть состояла в реализации **правила сравнения** и передачи его в сортировку.

1. **Переворот числа**

   - Итеративно вычисляется rev — переворот числа v, изначально $rev = 0$.
   - Если $v = 0$, то прекращаем вычисление.
   - Иначе:
     - увеличиваем rev в 10 раз.
     - прибавляем к rev последнюю цифру числа v (остаток от деления на 10).
     - целочисленно делим v на 10.
     - переходим к следующей итерации вычисления.

   Итоговая асимптотика: $O(log(v))$ для заданного числа v.

   Совет: чтобы оригинальное значение v не потерялось после вычисления, советуем вынести переворот числа в отдельную функцию с передачей параметра по значению (стандартный способ передачи для целых чисел).

2. **Подготовка данных**

   Создадим новый массив b, где элемент b_i будет содержать в себе информацию про

   - Оригинальное значение a_i;
   - Перевернутое представление числа rev;
   - Позицию в массиве i.

   Важно: вычисление перевернутого представления при каждом сравнении понизит общую эффективность алгоритма до $O(N \cdot log(N) \cdot log(max(v)))$.

3. **Сортировка по правилу**

   C++, Python:

   В данных языках для структур «динамический массив», «кортеж» и «строка» по умолчанию применяется встроенное правило **лексикографического сравнения**:

   (a) Элементы сравниваются по очереди в порядке итерации.

   (b) Сравнение прекращается на первом неравенстве элементов в одинаковых позициях.

   (c) Результат найденного неравенства является результатом всего сравнения.

   Поэтому предлагается сохранить данные в структуру «динамический массив» / «кортеж» в **следующем порядке**:

(a) Перевернутое представление числа;

(b) Отрицательное значение позиции в массиве;

(c) Оригинальное значение элемента массива.

В таком случае встроенная сортировка автоматически переставит данные в нужном порядке.

Дополнительная информация:

- Динамический массив:
  https://ru.algorithmica.org/cs/basic-structures/vector/
- Кортежи в C++:
  https://ru.algorithmica.org/cs/basic-structures/array/
- Лексикографический порядок в конспектах ИТМО:
  https://shorturl.at/yN049

Java:

Простейшим способом будет описать собственный **класс** с данными, после чего сделать одно из двух:

(a) либо описать у класса интерфейс **Comparable**, описывающий правило сравнения для объектов данного класса;

(b) либо передать в метод сортировки объект типа **Comparator** (в Java 8+ заменяется на лямбда-функцию), описывающий правило сравнения для объектов класса в рамках заданной сортировки.

Дополнительная информация:

- Comparable в Java docs:
  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/lang/Comparable.html
- Comparator в Java docs:
  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/Comparator.html
- Примеры использования на Baeldung:
  https://www.baeldung.com/java-comparator-comparable

Задача F. Чего-то не хватает...

Формальная **постановка задачи**: найдите значение массива, имеющее минимальную *частоту встречаемости* в массиве.

Определение: *частота (встречаемости)* f значения v в массиве a — количество позиций массива i таких, что $a_i = v$.

1. **Вычисление частот встречаемости**

   Воспользуемся структурой данных «словарь» («ассоциативный массив»):

   - *ключом* будет являться значение v;
   - *значением* — частота значения f.

   Обработаем все элементы массива a_i:

   - Если a_i нет в словаре — положим соответствие $(a_i, 1)$.

- Иначе запросим из словаря текущую частоту f, после чего положим соответствие $(a_i, f + 1)$.

Советы:

C++:

При **отсутствии соответствия** операция

```
имя_словаря[v]
```

кладет соответствие **по умолчанию**.

Для целых чисел значение по умолчанию равно 0.

Поэтому для обновления частоты можно использовать синтаксис

```
имя_словаря[v]++
```

Дополнительная информация:

- Словарь *unordered_ map* на основе **хеш-таблицы**
  https://en.cppreference.com/w/cpp/container/unordered_map
- Словарь *map* на основе **дерева поиска**
  https://en.cppreference.com/w/cpp/container/map

Java:

Начиная с Java 8, словари обладают методом *getOrDefault*.

Поэтому для обновления частоты можно использовать синтаксис

```
int f = имяСловаря.getOrDefault(v, 0);
имяСловаря.put(v, f + 1);
```

Дополнительная информация:

- Интерфейс словаря *Map*
  https://docs.oracle.com/en/java/javase/20/docs/api/java.base/java/util/Map.html
- Словарь *HashMap* на основе **хеш-таблицы**
  https://www.baeldung.com/java-hashmap
- Словарь *TreeMap* на основе **дерева поиска**
  https://www.baeldung.com/java-treemap

Python:

(a) Класс **Counter** модуля *collections* осуществляет вычисление частоты встречаемости всех значений массива.

(b) Класс **defaultdict** модуля *collections* обрабатывает отсутствие соответствия аналогично словарям в C++.

(c) Метод *get* класса **dict** работает аналогично методу *getOrDefault* словарей в Java.

Дополнительная информация:

- Словарь *dict* на основе **хеш-таблицы** в Python docs
  https://shorturl.at/ajp46
- Модуль *collections*
  https://docs.python.org/3/library/collections.html

2. **Поиск значения с минимальной частотой**

   - *Переберём* все соответствия (v, f) в словаре частот и найдём минимальную частоту f_{min}.
   - Повторно переберём все соответствия (v, f) и запомним значение v, для которого $f = f_{min}$.

   Перебор всех соответствий словаря является **стандартным функционалом**. Изучите **документацию** для используемого вами словаря.

Задача G. Есть ли сдача?

Формальная **постановка задачи**: найти наименьшую сумму $|x|+|y|$ среди решений *диофантова* уравнения $A \cdot x + B \cdot y = N$.

Определение: *Диофантовым* называют уравнение с **целочисленным** значением неизвестных переменных.

1. **Теоретическая информация**

   - Пусть G — **наибольший общий делитель** чисел A и B.
   - Любая линейная комбинация чисел (A, B)
     $A \cdot x + B \cdot y$
     делится нацело на G.
   - Для любых чисел (A, B) **существуют** целые числа (x, y) такие, что
     $A \cdot x + B \cdot y = G$.
     Поиск пары (x, y) осуществляет **расширенный алгоритм Евклида** за $O(log(min(A, B))$.
   - Если целые числа (x, y) удовлетворяют равенству
     $A \cdot x + B \cdot y = N$,
     то данному равенству удовлетворяют и все пары чисел
     $(x + k \cdot \frac{B}{G},\ y - k \cdot \frac{A}{G})$, где k — любое целое число.

   Дополнительная информация

   - Расширенный алгоритм Евклида на E-maxx
     https://e-maxx.ru/algo/extended_euclid_algorithm
   - Расширенный алгоритм Евклида на Алгоритмике
     https://ru.algorithmica.org/cs/modular/extended-euclid/
   - Диофантовы решения $A \cdot x + B \cdot y = N$ на E-maxx
     https://e-maxx.ru/algo/diofant_2_equation.

2. **Поиск решения с минимальной суммой модулей**

   Пусть $A \geqslant B$.

   Утверждение:

   Оптимальная сумма достигается:

   - либо при наименьшем неотрицательном y;
   - либо при наибольшем отрицательном y.

   Доказательство:

   Рассмотрим пару (x, y).

   Обозначим через (xn, yn) пару $(x + \frac{B}{G}, y - \frac{A}{G})$.

(a) (x, xn, y, yn) **одного знака**
Так как $A \geqslant B$, то $\frac{B}{G} - \frac{A}{G} \leqslant 0$.
Соответственно для $x \geqslant 0$ и $yn \geqslant 0$ выполняется $|x| + |y| \geqslant |xn| + |yn|$.
Аналогично для $y \leqslant 0$ и $xn \leqslant 0$ выполняется $|x| + |y| \leqslant |xn| + |yn|$.

(b) (x, xn) одного знака, (y, yn) одного знака, (x, y) **различных знаков**
В таком случае разница между суммой модулей равна $\frac{B}{G} + \frac{A}{G}$.
Для $xn \leqslant 0$, $yn \geqslant 0$ выполняется $|x| + |y| \geqslant |xn| + |yn|$.
Для $xn \geqslant 0$, $yn \leqslant 0$ выполняется $|x| + |y| \leqslant |xn| + |yn|$.

(c) **Пограничные** случаи
Можно показать, что все пограничные и неописанные выше случаи являются:

- либо переходом от 1 к 2 или обратно;
- либо переходом от наименьшего неотрицательного y к наибольшему отрицательному y или обратно.

3. **Решение**:

- Переходим к $A \geqslant B$.
- Находим любое решение (x, y) уравнения $A \cdot x + B \cdot y = N$ с помощью **расширенного алгоритма Евклида**.
- Вычисляем необходимое количество шагов $K = y \ div \ \frac{A}{G}$ (целочисленное деление).
- Переходим к решению $(xn, yn) = (x + K \cdot \frac{B}{G}, y - K \cdot \frac{A}{G})$.
- **Оптимальный ответ** достигается на одном из трёх решений:
  (a) (xn, yn);
  (b) $(xn + \frac{B}{G}, yn - \frac{A}{G})$;
  (c) $(xn - \frac{B}{G}, yn + \frac{A}{G})$.

  Уточнение: на самом деле всегда достаточно рассмотреть **два решения из трёх**, но это зависит от знака yn.

Итоговая асимптотика: $O(log(min(A, B)))$ на поиск тройки (G, x, y) и $O(1)$ на последующие вычисления.

Задача H. Вместе навсегда

Формальной постановки задачи не будет, так как проще всего рассуждать и решать задачу в терминах **оригинального условия**.

1. **Краткое изложение условия**

Есть две стороны: A-сторона с A людьми на ней, B-сторона с B людьми на ней.

Между сторонами есть N двусторонних порталов, каждый обладает прочностью f_i.

Портал уничтожается, если через него f_i раз прошел человек (в любую из сторон).

Требуется выяснить, можно ли **уничтожить все** порталы, **переместив всех** людей **на любую из сторон** (A или B).

2. **Основная идея**

Заметим, что количество порталов не имеет значения — важна лишь их **суммарная прочность** $S = \sum f_i$.

Поэтому можно заменить все порталы на **один большой портал** прочностью S.

Предположим, что всех людей необходимо переместить из A-стороны на B-сторону:

- Чтобы A человек смогли перейти, прочность портала S должна быть не меньше A.
- После того, как все люди с A-стороны перейдут, останется ровно $(S - A)$ прочности.
- Теперь все люди на стороне B, а значит любое перемещение на сторону A должно иметь пару — возврат на B.
- Отсюда следует, что оставшуюся прочность портала $(S - A)$ можно уменьшить за один раз на 2 единицы, отправляя любого человека **туда и обратно**.

Аналогичные рассуждения можно проделать для перемещения людей с B-стороны на A-сторону.

3. **Решение**

- Вычисляем суммарную прочность порталов $S = \sum f_i$.
- Переместить на B-сторону можно, если $S \geqslant A$ и $(S - A)$ — чётное.
- Аналогично переместить на A-сторону можно, если $S \geqslant B$ и $(S - B)$ — чётное.

Задача I. Налоговый вопрос

Формальная **постановка задачи**: найти количество вершин неориентированного графа, *степень* которых лежит в отрезке $[P; Q]$.

Определение: *степенью* вершины называют количество связанных с ней рёбер.

Решение:

- Заведём массив степеней вершин d, где изначально $d_v = 0$ для всех вершин v.
- Для каждого ребра (v, u) увеличим d_v и d_u на единицу.
- После обработки всех рёбер найдем количество таких вершин v, что $P \leqslant d_v \leqslant Q$.

Дополнительная информация:

- Основные определения теории графов на Codeforces EDU
  https://codeforces.com/edu/course/2/lesson/8